Journal of Ambient Intelligence and Humanized Computing (2023) 14:8693–8702
https://doi.org/10.1007/s12652-021-03625-8

ORIGINAL RESEARCH

A continuous RRT*-based path planning method for non-holonomic mobile robots using B-spline curves

S. A. Eshtehardian[1] · S. Khodaygan[1]

Received: 1 January 2021 / Accepted: 30 November 2021 / Published online: 21 January 2022

Abstract

Rapidly exploring random trees (RRT) are sampling-based approaches being widely applied for path planning of mobile robots. Since the output of these algorithms usually is a stream of discrete lines involving discontinuity at the linking points, kinematic constraints restrict the robot's movements. Consequently, robots may not pass discrete points in the path correctly. Hence, the using CAGD (Computer-Aided Geometry Design) curves can run simultaneously alongside those algorithms or may run after that to make a smooth path and that's the way in which non-holonomic constraints can be considered perfect and robots can be droved autonomously across them about the collision detection method which executed by the main sampling-based algorithm like RRT*. In this paper, an approach based on the combination of RRT* and B-spline is proposed for smoothing the path which is generated by RRT*-based algorithms, which are one of the most famous groups of algorithms in artificial intelligence. Some new functions are added to the outcome of the RRT* algorithm. To avoid collision in the generated path, some corrections are also provided. Finally, for illustrating the efficiency of the proposed method, the algorithm is implemented in the simulation environment of Webots® and for verification, the obtained results are compared and discussed.

Keywords Path planning · RRT · Algorithm · Smoothing algorithm · Non-holonomic mobile robots · B-spline curves

1 Introduction

Nowadays robots play a significant role in human life. Different kinds of robots such as arms, medical robots, and industrial robots are the most common ones. They usually perform autonomously to make various tasks easier for humans.

In autonomous robots, one of the most important issues which engineers and scientists have been trying to solve is motion planning. The term motion planning stands for generating a path for collision detection methods, whether it is feasible or not.

In general motion planning problems have two categories, mobile, and fixed robots. Mobile robots are which can move toward arbitrary directions, usually via their wheels. Using wheels for movement often restricts a robot's degree of freedom because of Kinematic and Dynamic constraints.

During the past few decades, different algorithms have been executed on mobile robots to have an obstacle-free path for them. Besides that, for more information, the different types of these algorithms have been reviewed in the literature (Iram Noreen et al. 2016). Also, in the past few years, classic algorithms have been proposed for path planning based on machine learning (Sombolestan et al. 2019).

Among those diverse motion planning algorithms, sampling-based methods, include PRM and RRT and similar algorithms, are the most practical ones because as said before in (Iram Noreen et al. 2016), these types of algorithms have some benefits over others like appropriation for high dimensional problems and less computational cost. In general, these algorithms find a possible solution in an infinite time running. Probabilistic Roadmaps (PRM) (Iram Noreen et al. 2016; E. Kavraki et al. 1996; M. Elbanhawi 2014) and RRT (Iram Noreen et al. 2016; S. M. Lavalle 1998; J. J. Kuffner 2000) and the modified versions of them are the most famous and practical members of this group. Algorithms in the PRM group are usually used in static environments which are structured. Also, they are some of the best solutions for multi-query problems. On the other hand, the RRT group is often

✉ S. Khodaygan
khodaygan@sharif.edu

[1] Department of Mechanical Engineering, Sharif University of Technology, Tehran, Iran

one of the best choices for both static and dynamic environments in which the problems are single-query.

In recent years, the RRT* algorithm, which is a developed version of RRT designed by Karaman and Frazzoli (S. Karaman et al. 2011), and its modifications such as RRT*-SMART (Jauwairia Nasir et al. 2013) have become suitable alternatives among different algorithms for motion planning problems. While RRT*-based algorithms usually are proper ones for mobile-robots motion planning, often they can't satisfy the dynamic or kinematic constraints of the non-holonomic mobile robots, especially wheeled mobile robots such as rescue ones. So, one of the gentle solutions which can smooth the generated path by the mentioned algorithms is utilizing an appropriate CAGD curve.

A similar method has been developed from the combination of Dynamic RRT and B-spline was designed in (Enzhong Shan et al. 2009). Then in (Kwangjin Yang et al. 2014), another Spline-Based RRT Path Planner for non-holonomic robots employing Bezier curve has been introduced. Another path smoothing using B-Spline for Car-Like robots has been introduced in (Mohamed Elbanhawi et al. 2015). Moreover, A new algorithm based on the mixing of RRT*-connect with the cubic Ferguson curve has been proposed in (Priyanka Sudhakara et al. 2017).

In (Iram Noreen et al. 2016) a future research plan for RRT*-group has been suggested which states this group of algorithms needs a method for smoothness based on CAGD curves such as a clamped B-spline or NURBS for non-holonomic mobile robots. As a result, the aim is to propose a new method that leads to the smoothness of RRT*-based manners more generally.

In this paper, the main aim is to introduce a new method derived from the combination of B-spline global interpolation and RRT*-based approaches for smoothing and appropriating the output of the algorithms for non-holonomic mobile robots.

2 Basic concepts

In the following subsections, to provide the required background for developing the proposed algorithm, the essential basic concepts are briefly reviewed.

2.1 B-splines

Since NURBS is the most comprehensive tool of CAGD curves which can generate each complex curve, it requires a lot of parameters for this intent. Hence, after many searches among CAGD tools, it is preferred to work with B-spline curves which can possess different degrees and knots and have pretty fewer parameters than NURBS. Besides that, these curves may be used for both approximation and interpolation (KunwooLee 1999). Moreover, they have local control on the curve unlike some other CAGD tools like Bezier (KunwooLee 1999).

Consider n given points each denoted by x_i. In general, a B-spline curve for approximating or interpolating such points is defined in Eq. 1 (KunwooLee 1999):

$$P(x) = \sum_{i=0}^{n} p_i N_{i,k}(x) \left(t_{k-1} \le x \le t_{n+1}\right) \tag{1}$$

where p_i's is the B-spline coefficient for each $N_{i,k}(x)$, t_j's are the elements of the knot vector demonstrated especially for each purpose.$N_{i,k}(x)$ is a blending function defined in Eq. 2 and Eq. 3 (KunwooLee, 1999):

$$N_{i,k}(x) = \frac{(x - t_i)N_{i,k-1}(x)}{t_{i+k-1} - t_i} + \frac{(t_{i+k} - x)N_{i+1,k-1}(x)}{t_{i+k} - t_{i+1}} \tag{2}$$

$$N_{i,1}(u) = \begin{cases} 1, t_i \le x \le t_{i+1} \\ 0, otherwise \end{cases} \tag{3}$$

2.2 RRT* algorithm

RRT* and its modifications are some of the most important and practical algorithms in robotic motion planning which was first developed by Karaman and Frazzoli (S. Karaman et al. 2011). In these algorithms, the output usually is one of the branches of a tree, similar to the classic RRT which was introduced earlier by Lavalle. Meanwhile, there is the main advantage that distinguishes RRT* from RRT and it is named probability completeness. Due to this attribute, RRT* can find the optimal path when the number of iterations goes toward infinite. Hence, RRT* is an appropriate choice. RRT* is presented in Algorithm 1 (BryanGuevara et al. 2018):

Algorithm 1	:	RRT*
1	:	Input: initial node x_{init}, max nodes $K \leftarrow N$
2	:	$V \leftarrow \{x_{init}\}; E \leftarrow \emptyset$;
3	:	for $i = 0$ to K do
4	:	$x_{rand} \leftarrow Sample(i)$
5	:	$x_{nearest} \leftarrow Nearest(V, x_{rand})$
6	:	$x_{new} \leftarrow Steer(x_{nearest}, x_{rand})$
7	:	$V \leftarrow V \cup \{x_{new}\}$
8	:	If ObstacleFree($x_{nearest}, x_{rand}$) then
9	:	$X_{near} \leftarrow Near(V, x_{new})$
10	:	$x_{nearest} \leftarrow Parent(X_{near}, x_{nearest}, x_{new})$
11	:	$E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$
12	:	$E \leftarrow Rewire(X_{near}, E, x_{new})$
13	:	end if
14	:	end for

Algorithm 1	: RRT*
15	: return (V, E)

About the functionality of RRT*, the goal and start points must be determined. Then, at each iteration, a random node in obstacle-free space is randomly chosen (Sample function) and the nearest node among the nodes of the graph will be selected if there are no obstacles between the x_{rand} and$x_{nearest}$, the distance between them must be checked. If the Euclidean distance between them was less than a constant amount, a direct line between them will be added to the graph, and x_{rand} is changed tox_{new}. Else, rely on the constant amount which was mentioned before, on the straight line between x_{rand} and $x_{nearest}$ at the distance which is equal to the constant amount from $x_{nearest}$, x_{new} will be selected (steer function as shown in Fig. 1).

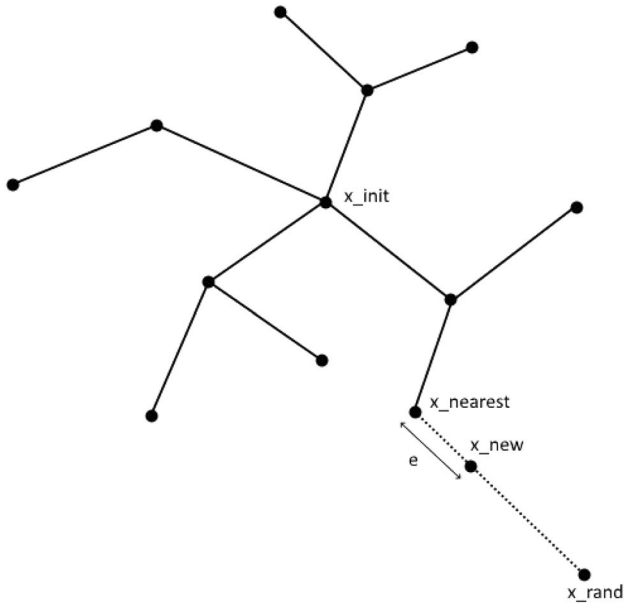


Fig. 1 Performance of steer function

Then, based on a predefined radius r_i with considering x_{new} at the center, near neighbors of x_{new} is determined by the near function. r_i is defined as denoted as follows (BryanGuevara et al. 2018):

$$r_i = min\left\{\left(\frac{\gamma \log n}{\zeta_d n}\right)^{1/d}, \eta\right\} \tag{4}$$

where η is a radius defined before, n is the number of nodes, and ζ_d is the volume of the space generated by r_i. Also, γ is a constant number which must satisfy as follows(BryanGuevara et al., 2018):

$$\gamma \geq 2^d\left(1+\frac{1}{d}\right)\mu\left(X_{free}\right) \tag{5}$$

where μ is equal to the volume of the collision-free space(In 2D assessment it means the area of the state space without the area of obstacles.)

Among the neighbors, one that after connecting to the random node has the minimum cost will be selected as the parent node as shown in Fig. 2. This process is done by the Parent function.

Essentially, the final step is going to do named rewiring by Rewire function. At this time, this approach checks the near nodes. If the path passing x_{new} and then one of the near nodes has less cost than the present path, the link between that node and its parent will be removed and the algorithm connects it to the x_{new}. This process shows in Fig. 3.

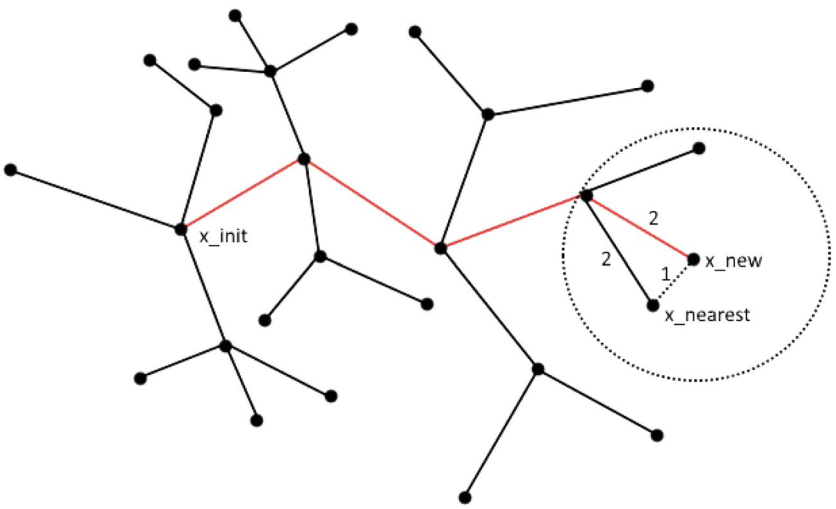


Fig. 2 Performance of the parent function

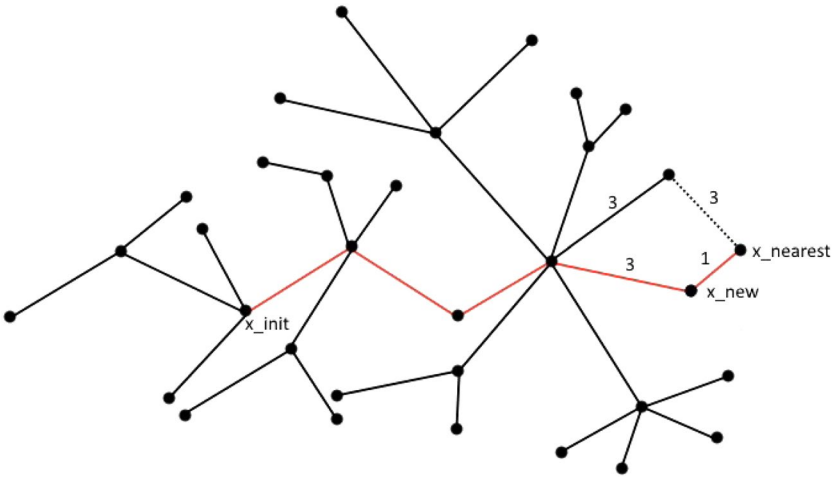


Fig. 3 Performance of rewiring function

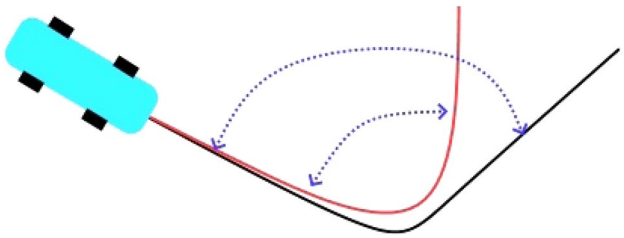


Fig. 4 Turning angle (radius) of a vehicle

2.3 Turning angle in path planning for non-holonomic mobile robot

Due to dynamic constraints, each non-holonomic vehicle (i.e., a non-holonomic mobile robot) has a limited angle (in another aspect radius) of turning, which, in turn, restricts the turning of that vehicle. Figure 4 shows an example of this property:

Referring to Fig. 4, the vehicle can turn most along the black path. However, the angle of the red path is less than the angle of black turning, so the vehicle can't go through it. In this paper, it is preferred to just simplify this concept, for more information, some modifications have been presented in the literature Ref. (Ettefagh et al. 2014).

3 Proposed method

In the proposed method, first of all, the RRT* algorithm (or a similar RRT*-based one) does the path generation method. The outcome consists of a lot of points which is conveyed by many lines as a graph structure. Then, according to the turning angle of the vehicle, which is a result of non-holonomic constraints, some of those points are eliminated to make a smoother and shorter path. After that, the outcome is changed by implementing B-spline interpolation on the result points. This method can smooth the path by generating a curve which the robot can go through it without main problems which are caused by non-holonomic constraints of the robot. The proposed method is shown in Algorithm 2:

Algorithm 2	:	Implementation of B-Spline interpolation on RRT*
1	:	Input: initial node x_{init}, max nodes $K \leftarrow N$
2	:	$V \leftarrow \{x_{init}\}; E \leftarrow \emptyset;$
3	:	for $i = 0$ to K do
4	:	$x_{rand} \leftarrow Sample(i)$
5	:	$x_{nearest} \leftarrow Nearest(V, x_{rand})$
6	:	$x_{new} \leftarrow Steer(x_{nearest}, x_{rand})$
7	:	$V \leftarrow V \cup \{x_{new}\}$
8	:	If ObstacleFree $(x_{nearest}, x_{rand})$ then
9	:	$X_{near} \leftarrow Near(V, x_{new})$
10	:	$x_{nearest} \leftarrow Parent(X_{near}, x_{nearest}, x_{new})$
11	:	$E \leftarrow E \cup \{(x_{nearest}, x_{new})\}$
12	:	$E \leftarrow Rewire(X_{near}, E, x_{new})$
13	:	end if
14	:	end for
15	:	$(V, E) \leftarrow Turningconsideration(V, E)$
16	:	Finalpath $\leftarrow Bspline(V, E)$
17	:	return Finalpath

In algorithm 2, two new functions are added to the traditional form of RRT*. In Turningconsideration, some of the nodes of RRT* are removed according to the available

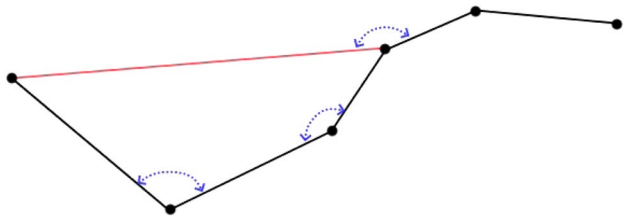


Fig. 5 Turningconsideration function

turning angle of the vehicle, and then in Bspline, the leftover nodes are interpolated. In Fig. 5, Turning consideration is implemented in a given RRT* graph (Fig. 5).

In Fig. 5, the red line is drawn because the turning angle between the two points on both ends of it is more appropriate for the vehicle in comparison to passing the two nodes among them. Similarly, other nodes that disturb the angle of the turning of the vehicle or robot are withdrawn, and then the B-Spline method is executed.

The turning consideration function can be defined according to each specific application because it is a general concept and related to the available turning angle or radius of the mobile robot. In this case, this function is defined as follows:" beginning from the start point, and for each point detect which two subsequent points after that can make a feasible angle with it, and then eliminated the points that are among them in the path, and then connect them and move to the selected point in the previous step." Again, the definition used in this article has been clarified in Fig. 5. It is worth noting that the line that connects the new points (i.e., in Fig. 5 the red line) should be completed in the collision-free space, else a terrible collision may be encountered. Its solution will be gone through in the following parts of this article.

The function Bspline consists of executing a B-Spline interpolation on the RRT* algorithm, according to the Fortran routine Parcur method for determining the coefficients of the B-Spline (Dierckx 1993). This will be elaborated on in the following.

Consider a dataset x_r which involves $m \times nmembers$. Let define x_r^* in which $r = 1,2,3\ldots,$ m as follows (Dierckx 1993):

$$x_{r,l}^* = a_l x_{r,l} + b_l \qquad r = 1,\ldots,m,\ l = 1,\ldots,n \tag{6}$$

And $s^*(x^*) = (s_1^*(x^*),\ldots,s_n^*(x^*))$ is the curve corresponding to it which is acquired from Parcur or Clocur and having B-spline coefficient $p_{i,l}^*$ in which i = -k,…,g. According to the normalization property which is noticed in (BryanGuevara et al. 2018), for achieving the B-spline curve $s(x) = (s_1(x),\ldots,s_n(x))$ which interpolates the data points belong to x_r must have the coefficient as follows:

$$p_{i,l} = \frac{p_{i,l}^* - b_l}{a_l}, i = -k,\ldots,g, l = 1,\ldots,n. \tag{7}$$

The shape of the curve may firmly be affected by choosing parameter values u_r. They can be chosen based on the problem's physical consideration (For instance, when someone wants to use this method for four-wheeled autonomous robots it is recommended to use the centripetal method). In this method, the chord length parametrization is used based on the experimental robot's kinematic constraints (Dierckx 1993):

$$u_1 = 0 \tag{8}$$

$$u_r = u_{r-1} + d_r \tag{9}$$

$$a = u_1, b = u_m \tag{10}$$

$$d_r = \|x_r - x_{r-1}\| \tag{11}$$

Using a low degree belongs to the B-spline aims to produce a wonderful collision avoidance method based on the interpolation of the B-spline. In other words, when the robot's dimensions are chosen appropriately, then the collision detection method will be used in RRT* because in contrast to approximation methods, in this interpolation the convex hull property is not applicable. As shown in Fig. 6, this

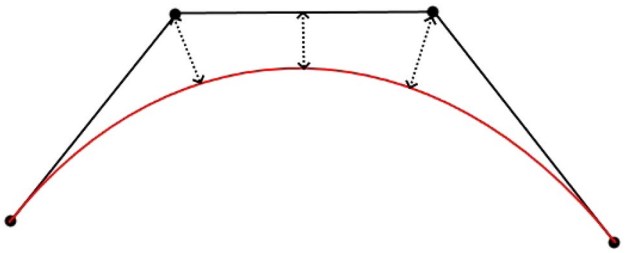


Fig. 6 Convex hull property

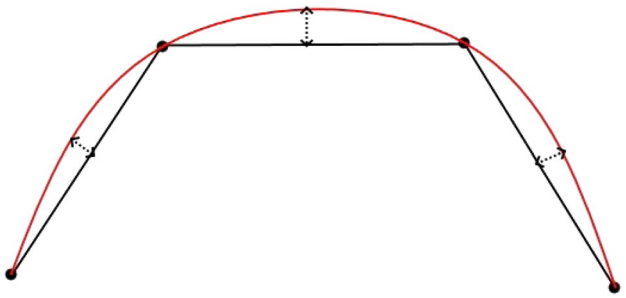


Fig. 7 Global interpolation

property may cause some problems in collision avoidance, but in Fig. 7 It is shown that interpolation usually doesn't deserve such problems and can produce a more confident path rather than approximation methods.

As a result, it is recommended that in this running the obstacle's dimension should be chosen in a way in which they are a little bigger than the real dimensions to make us ensure collision avoidance.

Furthermore, some recent methods have been proposed in the literature (S. L. Herbert et al. 2017; D. Fridovich-Keil et al. 2018; Sylvia Herbert et al. 2021; SL Herbert, 2020) in which some error bounds due to the types of path and obstacles are computed. Then, by calculating H-J reachability, some initial conditions can be available that if the mobile robot movement starts based on them, a safe and sound path planning without collision can be obtained.

4 Case study

For evaluating the proposed algorithm, it was decided to design some experimental simulations on the e-puck robot, which has been designed by GCtronics, in Webots application. e-puck is a two-wheeled robot that has 0.074 m diameter and 0.045 m height based on the Cyberbotics website as shown overtly in Fig. 8.

Also, a GPS module was installed on it because of the easier implementation of the proposed algorithm.

It worth to note that e-puck and some similar circular robots may own a capability of utilizing turning in their current location without changing their center of gravity's coordination. Regarding this, the robot can pass the discrete paths mentioned before, but still, it needs to stop on the nodes in the RRT*s output, and after turning them in the right direction they will be able to track the path generated by RRT* (or RRT). Therefore, our proposed method is still more efficacious, by which the robots resembling e-puck can continue their way without pause and faster.

The chosen environments consist of a $3m \times 3m$ field with some boxes as an obstacle (Fig. 9).

In order to show the application of our algorithm in various environments, there are three different experiments from which we have done will be shown there:

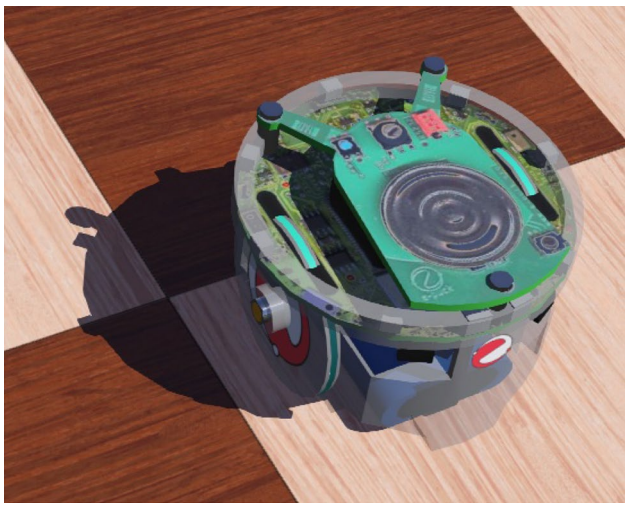
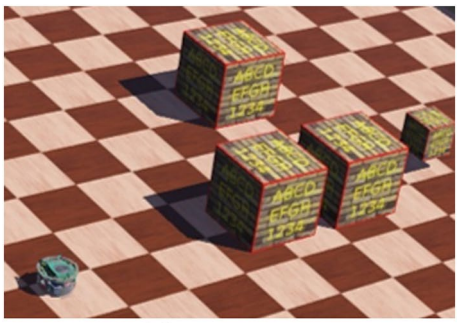
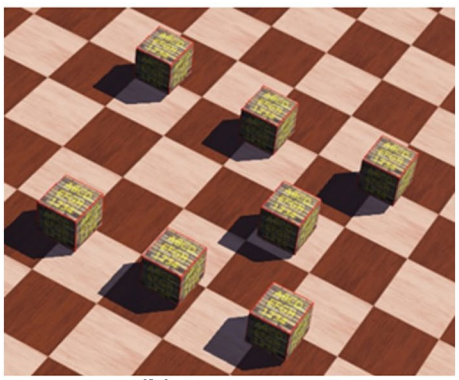


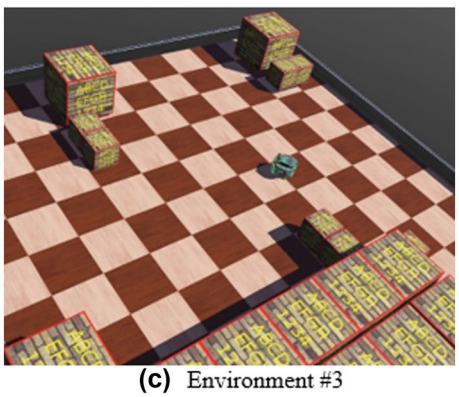
Fig. 8 A view of the e-puck robot



(a) Environment #1



(b) Environment #2



(c) Environment #3

Fig. 9 A view of the experiment environments

Environment #1: Four boxes are considered as obstacles which three of them is $0.15m \times 0.15m \times 0.15m$ and the fourth one in $0.075m \times 0.075m \times 0.075m$.

Environment #2: Four boxes are considered as obstacles and their dimensions are $0.075m \times 0.075m \times 0.075m$.

Environment #3: Twenty-four boxes are considered as obstacles and the fifteen of them have the dimensions $0.15m \times 0.15m \times 0.15m$, and also the dimensions of other nine boxes are $0.075m \times 0.075m \times 0.075m$.

For determining the path, this algorithm was run on the simulated map within python, and for having good collision avoidance obstacles dimensions were defined almost 33 percent bigger than the real as shown in Fig. 11 as blue rectangles. The 33 percent bigger means trying to create an allowance space between the real obstacles and simulated ones to omitting obstacle collision This allowance is based on the robot's dimensions and environment. As a result, it may be more or less. For example, for the more reliable design, it can be individually chosen more than the robot's radius, but in this experiment, the mentioned allowance seems appropriate (Fig. 10).

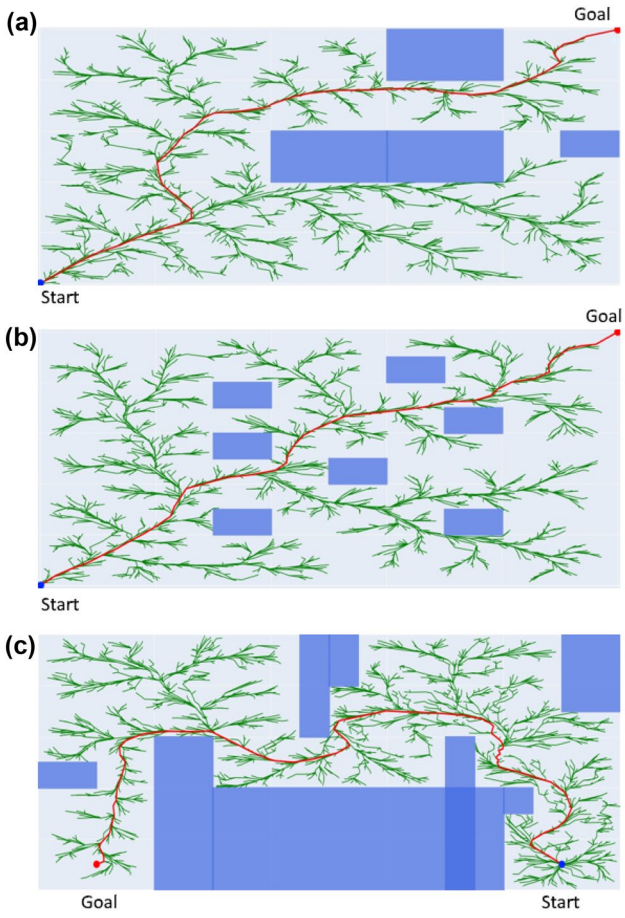


Fig. 10 The generated path through the RRT* algorithm: (**a**) Enviement #1 (**b**) Envirement #2 (**c**) Envirement #3

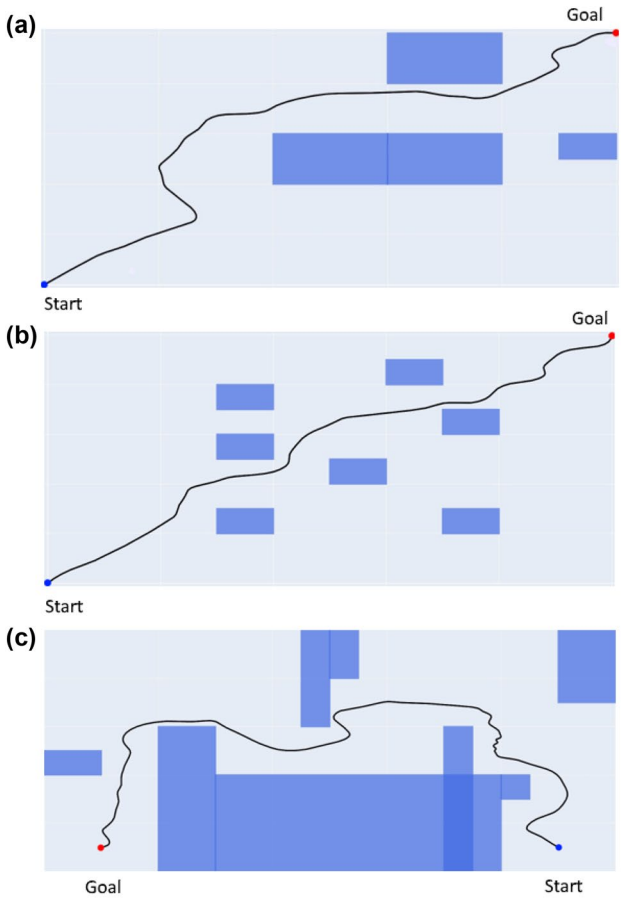


Fig. 11 The smoothed paths through the proposed algorithm: (**a**) Environment #1 (**b**) Environment #2 (**c**) Envirement #3

Figure 10 is the outputs of the RRT* algorithm in the defined maps where in the experiments 1 and 2 the blue points at the left bottom corner are the start points at $(0, 0)$ and the right top corners in red are the goal points at $(100cm, 100cm)$. In the environment 3, the blue point at the bottom right (90 cm, 10 cm) is the start point, and the red one at the bottom left is the goal point (10 cm, 10 cm). The red paths are the paths generated by RRT* which the robot must track.

Then the red paths are changed into the black ones by interpolating cubic B-spline on the red paths as shown in Figs. 11, 12.

After running the generated paths were tested on the e-puck robot in the prepared environment and the robot passed some of the paths successfully. But it might get into trouble with some parts of delicate paths, still according to the dynamic constraints. After all, if the approximate

length of the curves is calculated after using B-spline interpolation, it can be easily seen that it is has a few differences in comparison to the RRT* path before smoothing. For example, in experiment 1, the length of the curve is almost equal to 1.71 m; however, the length of the first path before smoothing was almost 1.69 m. Because of the nature of non-holonomic constraints, the robot could not trace the path generated by RRT* completely. Therefore, executing the B-spline function itself seems enough for the proposed method, regardless of the length of the path. However, a shorter and smoother path is preferred. So, this time Turningconsideration function is implemented first to make a shorter and smoother path and then the B-spline function. The final paths are shown in Table 1 and the results in Figs. 12, 13.

Comparing Figs. 13, 14, it can be observed that the paths after implementation of Turning consideration are smoother (the blue path is interpolated based on the B-Spline). In addition, due to Table 1, the length of the paths this time has been shorter, so the only problem in comparison to the classic RRT* is solved. Moreover, the problems may catch up with the robot in a delicate path due to the dynamic and kinematic constraints that will be puzzled out intelligently.

However, there is a fact in terms of the proposed algorithm which must be pointed out. In collision detection, as was mentioned before, it is better to assume the dimension of the obstacles a bit more than their real size (It is been done similarly in the simulation for about 33 percent bigger). That's because this algorithm still needs some developments in this way, and as shown in Fig. 14, the green points are some dangerous points that the robot may bump into the obstacles there because the obstacle-free space all of the environment without obstacles was completely considered. But, according to the size of the robot (its radius), the obstacles assumed bigger than their real size and so now there are no problems in this term (The obstacles have been shown completely in the previous Figures, such as Fig. 11). Please notice that as it has been said before, the B-Spline interpolation is a good manner for collision avoidance, and in Fig. 14 it overtly turns the boxes with allowance distance (the blue path).

5 Conclusions

In this paper, the proposed algorithm may be a perfect choice for a lot of mobile robots, but yet it isn't a comprehensive algorithm. It is strictly suggested engineers, students, and all of the users utilize this algorithm with SLAM methods simultaneously because it makes the collision detection process more trustable in dynamic environments. Also, basic RRT* has been developed a lot, and using new modifications of it such as RRT*-smart as the basic algorithm is proposed before smoothness because they usually take less time for running and generate better paths. Finally, selecting

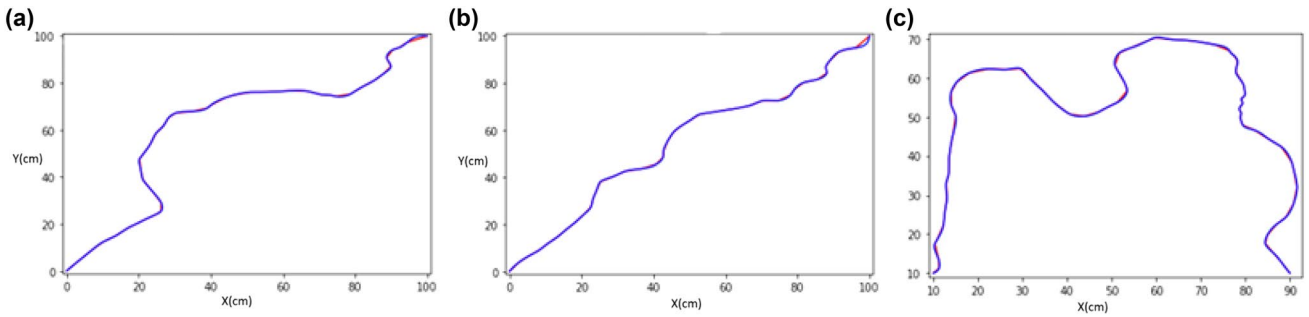


Fig. 12 The interpolated RRT* without Turningconsideration: (**a**) Enviement #1 (**b**) Envirement #2 (**c**) Envirement #3 (The red lines are stright lines connecting points, and the blue curve is the interpolated Bspline)

Table 1 A comparison between using Turning consideration and pure B- Spline

Algorithm	B-spline with RRT*	B-spline with RRT* with Turning consideration
Generated path length- Enviement #1 (cm)	170.92	149.78
Generated path length- Envirement #2(cm)	152.05	148.91
Generated path length- Envirement #3(cm)	203.77	200.49

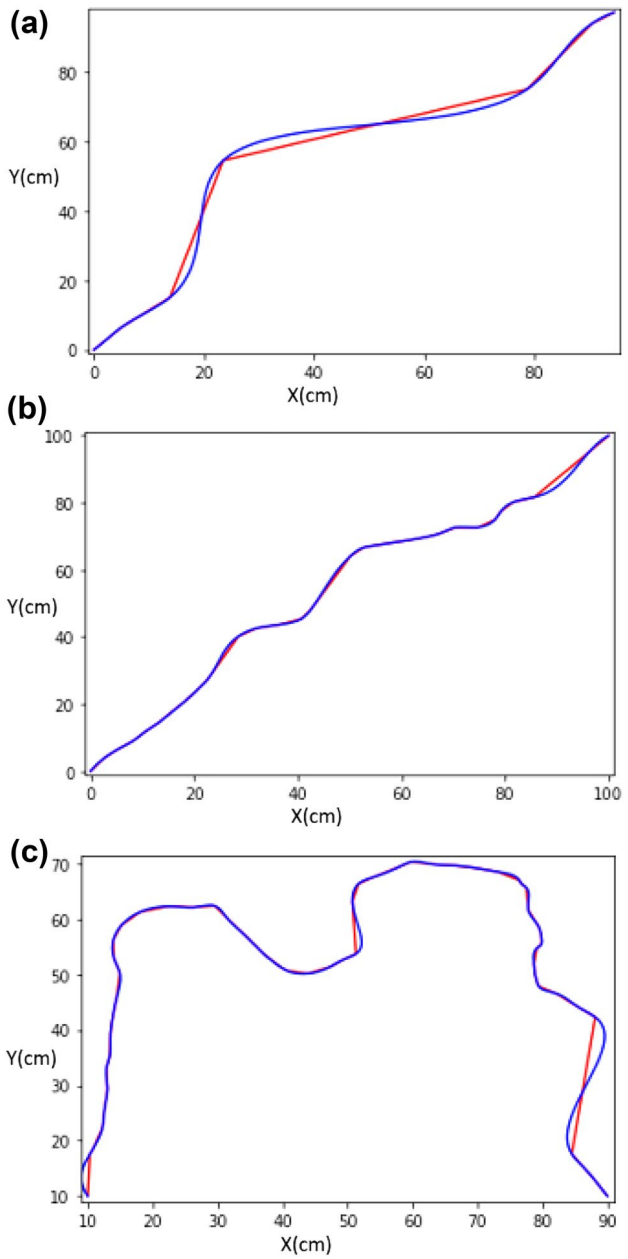


Fig. 13 The interpolated RRT* after Turningconsideration: (**a**) Enviement #1 (**b**) Envirement #2 (**c**) Envirement #3 (The red lines are stright lines connecting points, and the blue curve is the interpolated Bspline)

the parameters of the B-spline must be dependent on the dynamic or kinematic constraints of the robot. For instance, as has been mentioned before for four-wheeled mobile robots, it is recommended to use a centripetal method for determining parameters to the reduction of centrifugal forces during steering.

Finally, as a perfect recommendation for pursuing this research and completing it, as said before in the final part of the proposed method (part 3), it is highly preferred to

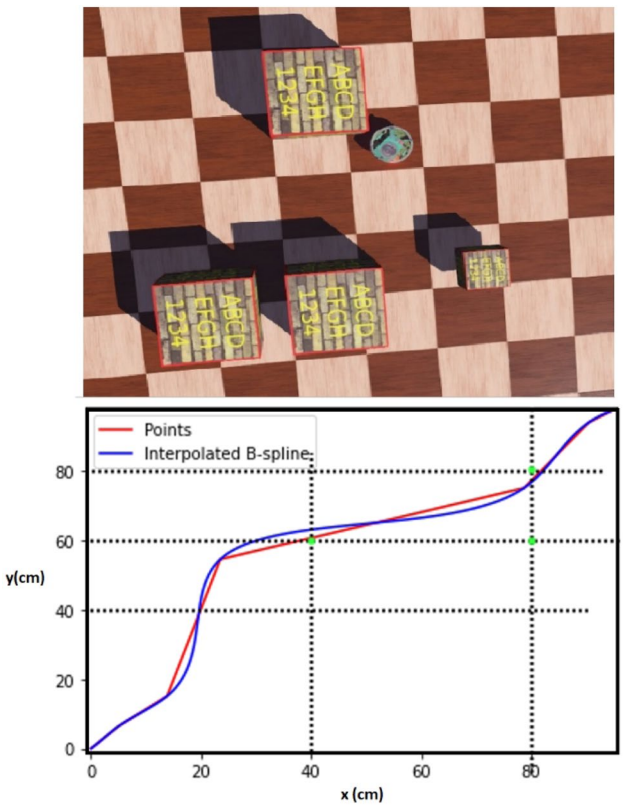


Fig. 14 The performance of the proposed method in collision detection

combine the proposed method with some collision avoidance methods derived from H-J reachability and FaS-Tracking in (S. L. Herbert et al. 2017; D. Fridovich-Keil et al. 2018; Sylvia Herbert et al. 2021; SL Herbert 2020) to have a reliable collision avoidance method besides the proposed method in this paper.

Supplementary Information The online version contains supplementary material available at https://doi.org/10.1007/s12652-021-03625-8.

References

Bryan G (2018) An overview of the class of rapidly-exploring random trees. Utrecht University

Dierckx P (1993) Curve and surface fitting with splines

Elbanhawi M, Simic M (2014) Sampling-based robot motion planning: a review survey. IEEE Access 2:56–77

Enzhong SBD (2009) An dynamic RRT path planning algorithm based on B-spline. second international symposium on computational intelligence and design

Ettefagh MM-J (2014). Optimal synthesis of four-bar steering mechanism using AIS and genetic algorithms. J Mech Sci Technol 2351–2362.

Fridovich-Keil D, Herbert SL, Fisac JF, Deglurkar S, Tomlin CJ (2018) Planning, fast and slow: a framework for adaptive real-time safe trajectory planning, 2018 IEEE international

conference on robotics and automation (ICRA), Brisbane, QLD, pp. 387–394, https://doi.org/10.1109/ICRA.2018.8460863
Herbert SL, Chen M, Han S, Bansal S, Fisac JF, Tomlin CJ (2017) FaSTrack: a modular framework for fast and guaranteed safe motion planning 2017 IEEE 56th annual conference on decision and control (CDC), Melbourne, VIC, pp. 1517–1522, https://doi.org/10.1109/CDC.2017.8263867
Herbert SL (2020) Safe real-world autonomy in uncertain and unstructured environments. University of California
Iram NAK (2016) Optimal path planning using rrt*based approaches: a survey and future directions. Internat J Adv Comput Sci Appl (IJACSA) 7:11
Jauwairia Nasir FI (2013) RRT*-smart: a rapid convergence implementation of RRT*. Int J Adv Rob Syst. https://doi.org/10.5772/56718
Karaman S, Frazzoli E (2011) Sampling-based algorithms for optimal motion planning. Int J Rob Res 30:846–894
Kavraki E, Svestka P, Latombe JC, Overmars M (1996) Probabilistic roadmaps for path planning in high dimensional configuration spaces. IEEE Trans Robot Autom 12:566–580
Kuffner JJ, Lavalle SM (2000) RRT-connect: an efficient approach to single-query path planning, in Proceedings of IEEE international conference on robotics and automation (ICRA), pp. 1–7.
Kunwoo L (1999) Principles of CAD/CAM/CAE. PrenticeHall
Kwangjin Yang SM (2014) Spline-based RRT path planner for non-holonomic robots. J Intell Rob Syst. https://doi.org/10.1007/s10846-013-9963-y
Lavalle SM (1998) Rapidly-exploring random trees: a new tool for path planning
Mohamed EMS (2015) Continuous path smoothing for car-like robots using B-spline curves. J Intel Robot Syst
Priyanka SVG (2017) Optimal trajectory planning based on bidirectional spline-RRT∗ for wheeled mobile robot. Third international conference on sensing, signal processing and security (ICSSS), https://doi.org/10.1109/SSPS.2017.8071566
Sombolestan SM, Rasooli A, Khodaygan S (2019) Optimal path-planning for mobile robots to find a hidden target in an unknown environment based on machine learning. J Ambient Intell Humaniz Comput 10(5):1841–1850
Sylvia H, Jason JC, Suvansh S, Marsalis G, Koushil S, Claire JT (2021) Scalable learning of safety guarantees for autonomous systems using hamilton-jacobi reachability 2021 IEEE international conference on robotics and automation (ICRA), Xi'an, arXiv : 2101.05916v2